



Escuela Técnica Superior de
Ingeniería Informática

SEGURIDAD EN SISTEMAS INFORMÁTICOS Y EN INTERNET

PAI 1. INTEGRIDOS - VERIFICADORES DE INTEGRIDAD EN EL ALMACENAMIENTO Y TRANSMISIÓN PARA ENTIDAD FINANCIERA

SECURITY TEAM 2

Alejandro Fernández Orozco
Alejandro Morilla García

1. Verificador de Integridad

1.1 Problema planteado

En cualquier entidad financiera es fundamental asegurarse de que las transacciones realizadas sean de forma íntegra y segura. Uno de los problemas más comunes suele ser el ataque “Man in the Middle”, como su propio nombre indica es alguien que intercepta los mensajes mientras se envían, teniendo así la capacidad tanto de leerlos como de modificarlos a su antojo para su propio beneficio.

Debido a este tipo de situaciones surge la necesidad de implementar herramientas que nos informen con la mayor rapidez posible de este tipo de fraudes para poder informar de la integridad y seguridad de los datos recibidos.

1.2 Decisiones

- Desarrollo sobre python dado que sus librerías facilitan la implementación.
- El proceso de verificación se realizará cada vez que enviemos una transacción, y debe almacenarse un log para guardar el identificador de cada transacción

1.3 Soluciones aportadas

Siguiendo la Política de Seguridad que se nos exigía, la solución más conveniente ha sido simular una conexión cliente-servidor donde podamos enviar la transacción, y cuando la reciba el servidor, poder saber si ha llegado íntegra o si ha sufrido un ataque de Replay por un “Man in the middle”.

2. Desarrollo, pruebas y análisis de los resultados

El programa que vamos a detallar ha sido desarrollado en lenguaje Python, y se ha usado Visual Studio Code como entorno de desarrollo del programa. Hemos elegido el lenguaje Python porque nos proporciona una gran cantidad de módulos relacionados con la seguridad informática.

El programa consta de 3 archivos:

- **clientsocket.py**: En este archivo se encuentra el código desarrollado para que un cliente se conecte al servidor y se pueda simular la transacción bancaria. Cuenta con mensajes formateados, la información de la cuenta, cantidad enviada y una HMAC para garantizar la integridad de los datos.
- **serversocket.py**: En este archivo se encuentra el código desarrollado de un servidor que escucha conexiones entrantes en una dirección IP y puerto específico. Cuando recibe un mensaje del cliente, calcula un HMAC para ese mensaje y verifica su integridad. También detecta intentos de repetir mensajes y los registra con el nonce (identificador). Luego, envía una respuesta al cliente indicando si el mensaje es válido o si hubo problemas de integridad.
- **mitm.py**: En este archivo se encuentra el código desarrollado para una simulación de un ataque de “Man in the middle”. El atacante MITM intercepta la comunicación entre un cliente y un servidor real. El cliente se conecta al atacante, quien a su vez se conecta al servidor real. El atacante recibe mensajes del cliente, decide si modificarlos y, si lo hace, genera un nuevo HMAC y envía el mensaje modificado al servidor real. Si no modifica el mensaje, lo reenvía tal como está al servidor. El atacante también reenvía la respuesta del servidor real al cliente.

A continuación pasamos a explicar en detalle cada una de las funciones que componen estos archivos.

Las funciones principales del archivo **clientsocket.py**:

- **generar_nonce()**: Crea un valor de nonce de 8B y lo representa como cadena hexadecimal. Se ha utilizado la biblioteca “secreta” para generar una secuencia aleatoria no predecible.

- **calcular_hmac(mensaje, clave):** Calcula la HMAC para el mensaje pasado, utilizando la clave dada como argumento. Dentro de la función se define el algoritmo que utilizamos para generar el hash, en este caso es 'sha256'. Finalmente la almacenamos en la variable `hmac_hex`, representada como cadena hexadecimal para que sea legible y poder verificar su integridad.
- **with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:** Realizamos la conexión TCP a la dirección y puertos especificados. Entramos en bucle infinito con la comunicación en abierto:
 - El cliente solicita al usuario que se ingresa la "cuentaOrigen", "cuentaDestino" y "cantidad".
 - Se crea un mensaje que incluye la información que ha introducido el usuario y el nonce.
 - Creamos la HMAC de ese mensaje utilizando la clave acordada entre cliente y servidor, y la concatenamos junto al mensaje, para que todo esto se envíe al servidor, codificados en utf-8.
 - El bucle finalizará (la conexión) cuando el usuario ingrese "exit".

En cuanto a las funciones del **serversocket.py**, este primero abre la conexión para un servidor con una dirección y puerto dado y luego espera la conexión de un cliente:

- **calcular_hmac(mensaje, clave):** Calcula la HMAC para el mensaje pasado, utilizando la clave dada como argumento. Dentro de la función se define el algoritmo que utilizamos para generar el hash, en este caso es 'sha256'. Finalmente la almacenamos en la variable `hmac_hex`, representada como cadena hexadecimal para que sea legible y poder verificar su integridad.
- **guardar_nonce_en_archivo(archivo, nonce):** Esta función guarda en el archivo `nonce.txt` el nonce(identificadores de mensaje) que va dentro del mensaje.
- **with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:** Abrimos la conexión TCP con la dirección y puerto especificados. Entramos en un bucle infinito en el que realizamos la conexión y se queda en modo escucha esperando que algún cliente se conecte. Cuando se realiza la conexión se espera un mensaje del cliente y al recibir el mensaje, se descompone en 4 partes y se realizan diferentes operaciones para comprobar la integridad del mensaje.

En cuanto a la integridad encontramos 3 opciones:

- **Fallo de Integridad:** El mensaje se ha modificado pero al no tener la clave no coincide, por lo tanto es un fallo de integridad.
- **Fallo de Integridad por Replay Attack:** El mensaje enviado ha sido enviado anteriormente, por lo tanto es exactamente igual. Como el nonce coincide no es aceptado y se realiza un fallo por Replay Attack
- **Conservación de la Integridad:** Todo coincide perfectamente y ha sido comprobado, entonces se produce un acierto y el servidor responde con que se ha conservado la integridad.

Para finalizar explicamos las funciones que principalmente componen el script mitm.py:

- **calcular_hmac(mensaje, clave):** Realiza la misma función que calcular_hmac de serversocket.py. Utiliza una clave distinta al cliente y servidor para así simular un ataque en el que el mitm no conoce la clave, detectando así el fallo de integridad el servidor.
- **obtener_datos_mensaje(nonce):** Esta función permite al atacante ingresar datos de un nuevo mensaje. El atacante ingresa la cuenta de origen, la cuenta de destino, la cantidad y un nonce (número único). La función crea un mensaje con estos datos y el nonce proporcionado como argumento y lo devuelve como una cadena.
- **main():** Se encarga de controlar el flujo de ejecución. La función principal del programa es controlar el flujo de ejecución. Inicia una conexión de socket en el atacante MITM, espera la conexión del cliente, y luego establece una conexión saliente con el servidor real. Dentro de un bucle, el programa escucha los mensajes del cliente, los muestra, y ofrece al atacante la opción de modificar el mensaje. Si el atacante decide modificar el mensaje, se utiliza la función 'obtener_datos_mensaje' para obtener los nuevos datos y se recalcula el HMAC con 'calcular_hmac'. Luego, el mensaje modificado se envía al servidor real. Si no se modifica el mensaje, el mensaje original se envía tal como está. El programa también reenvía las respuestas del servidor real al cliente para que ambos extremos reciban las respuestas.

A continuación procedemos a mostrar las pruebas de ejecución de código junto con una explicación analizando la misma:

Al realizar el inicio de conexión del servidor, se muestra lo siguiente:

```
PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\serversocket.py
Server listening on 127.0.0.1:3030
```

Una vez que está en modo escucha, se mantendrá esperando que cualquier cliente se conecte, una vez conectado podrá enviar mensajes

```
PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\serversocket.py
Server listening on 127.0.0.1:3030
Connected by ('127.0.0.1', 54246)
```

Una vez que el servidor recibe mensaje, como hemos explicado anteriormente pueden ocurrir 3 opciones:

- En la primera opción no hay fallos, se realiza el intercambio manteniendo su integridad.

```
PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\serversocket.py
Server listening on 127.0.0.1:3030
Connected by ('127.0.0.1', 54246)
Received from client: es40,es50,200,1750f6a16bbf3ce8;cae5b38f035ae11f63c8d4ded15ec6ce0632b0d722d922c8d41d9b66b6536376
Se ha conservado la integridad
```

- En la segunda opción ocurre un fallo por integridad ya que el mensaje ha sido cambiado y los hashes no coinciden.

```
PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\serversocket.py
Server listening on 127.0.0.1:3030
Connected by ('127.0.0.1', 54449)
Received from client: es60,es70,500,0;d7e1ab20fb69c7e8b7dc41b3e1433904e99205103b6f9e66ea3829ee9bccf319
Fallo de Integridad
```

- En la tercera opción se produce un fallo por replay attack en que son los mismos mensajes pero el nonce al no coincidir, da fallo.

```
PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\serversocket.py
Server listening on 127.0.0.1:3030
Connected by ('127.0.0.1', 54449)
Received from client: es60,es70,500,0;d7e1ab20fb69c7e8b7dc41b3e1433904e99205103b6f9e66ea3829ee9bccf319
Fallo de Integridad
Received from client: es10,es20,400,41fcae64ee4bc827;96edbfedf067d9ece7f3599a988d3e020ef429a60ee40dca8712c03899d16344
Fallo de integridad por Replay Attack
```

A continuación mostramos como es la interfaz del cliente, el usuario introduce la cuenta de origen, la de destino y la cantidad deseada a enviar en la transacción :

```
PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\clientsocket.py
Connected to server at 127.0.0.1:3031
Ingrese la cuenta origen: es60
Ingrese la cuenta destino: es80
Ingrese la cantidad: 300
```

Desde el lado del servidor recibimos el mensaje de la siguiente forma: cuentaOrigen, cuentaDestino, cantidad, hmac; nonce , y un mensaje con una de las posibles opciones descritas anteriormente sobre el mensaje, en este caso, ha conservado su integridad.

```
PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\serversocket.py
Server listening on 127.0.0.1:3030
Connected by ('127.0.0.1', 54246)
Received from client: es40,es50,200,1750f6a16bbf3ce8;cae5b38f035ae11f63c8d4ded15ec6ce0632b0d722d922c8d41d9b66b6536376
Se ha conservado la integridad
```

Por otro lado, tenemos el archivo nonces.txt, donde se van registrando automáticamente todos los nonces que nos van llegando de las diferentes transacciones. de esta manera, podemos comprobar en ese fichero si algún nonce nuevo, esta repetido, y por lo tanto indicaría que nos encontramos antes un ataque de Replay.

```
src >  nonces.txt
1  NONCES REGISTRADOS
2  d460c17d7c7b63d3
3  bd66d735cdf064e3
4  847fcd1f1783675c
5  4d429699886a280c
6  8bcce3f27e265014
```

Hemos creído conveniente, crear otro cliente que simule al “man in the middle” para probar a estar escuchando la comunicación, y copiar una

transacción e intentar a reenviarla a ver qué sucede. Este cliente debe iniciarse antes de iniciar el cliente, es decir iniciamos servidor, iniciamos mitm e iniciamos cliente. Seguimos el mismo procedimiento, el cliente envía la transacción al servidor pero el mitm la intercepta:

<pre>PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\mitm.py Man-in-the-middle listening on 127.0.0.1:3031 Conexión desde el cliente: ('127.0.0.1', 55130) Mensaje recibido del cliente: es60,es50,300,47cf24086ec0a697 es60,es50,300,47cf24086ec0a697;d6fe5c99be86bc77c855136c282dad017b20a03d0fee48fb24ddb7c1c0889e5c ¿Quieres modificar el mensaje recibido? 1.Si 2.No</pre>	<pre>PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\clientsocket.py Connected to server at 127.0.0.1:3031 Ingrese la cuenta origen: es60 Ingrese la cuenta destino: es50 Ingrese la cantidad: 300 Ingrese la cuenta origen: </pre>
--	--

El agente mitm recibe el mensaje enviado por el cliente y tiene la opción de modificarlo o no. En el mejor de los casos, el mitm no lo modifica. Esto hace que reenvíe el mensaje al servidor tal cual le llega, lo que no provocaría ningún fallo de integridad al haber recibido el mensaje correctamente.

<pre>PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\serversocket.py Server listening on 127.0.0.1:3030 Connected by ('127.0.0.1', 55131) Received from client: es60,es50,300,47cf24086ec0a697;d6fe5c99be86bc77c855136c282dad017b20a03d0fee48fb24ddb7c1c0889e5c Se ha conservado la integridad </pre>	<pre>PS C:\Users\aleja\SSII\PAI-01\PAI1-SSII\src> py .\mitm.py Man-in-the-middle listening on 127.0.0.1:3031 Conexión desde el cliente: ('127.0.0.1', 55130) Mensaje recibido del cliente: es60,es50,300,47cf24086ec0a697 es60,es50,300,47cf24086ec0a697;d6fe5c99be86bc77c855136c282dad017b20a03d0fee48fb24ddb7c1c0889e5c ¿Quieres modificar el mensaje recibido? 1.Si 2.No 2 </pre>
--	---

Ahora vamos a ver qué ocurre en el caso de que el atacante quiera modificar el mensaje. El archivo mitm va a pedir que introduzca los datos que van a ser modificados, por lo que habría que introducirlos para seguir con el ataque.

```
Ingrese la cuenta origen: es40
Ingrese la cuenta destino: es50
Ingrese la cantidad: 100
```

```
Mensaje recibido del cliente: es40,es50,100,
a8e52679c1bcfabd
es40,es50,100,a8e52679c1bcfabd;7ccb69e306f27
322200ed678595c00137df0f6a09fb4de7925788d905
4b25965
¿Quieres modificar el mensaje recibido?
1.Si
2.No
1
Ingrese la cuenta origen: es40
Ingrese la cuenta destino: es85
Ingrese la cantidad: 100000
```

Introducimos los datos que queremos modificar, una vez hecho esto el mitm calcula la hmac del mensaje que introducimos pero lo hace con una clave distinta, ya que en este supuesto, el mitm no conoce la clave del servidor y del cliente. El mitm recopila los nuevos datos y genera de nuevo la hmac para mandarla al servidor. Una vez hecho esto, el servidor recibe el mensaje pero como el hmac está generado con una clave distinta nos detecta el error de integridad. Así vemos cómo protegernos de los ataques Man-In-The-Middle.

```
Received from client: es40,es85,100000,0;331
704e03cc966dd10a5ffe72b60821c10b74bd1707b393
6d846d3d2f0090a0e
Fallo de Integridad
```